

연구 논문

잘못된 오류 전제가 생성형 AI 의 Python 코드 수정 행동에 미치는 영향

김라연

2026 년 9 월

ALEPH 과제 10 · AI 와 함께 쓰는 첫 논문

초록

본 연구는 정상적으로 동작하는 Python 코드에 오류가 있다는 잘못된 전제를 제공할 때 생성형 AI의 코드 수정 행동이 달라지는지를 확인하는 것을 목적으로 한다. 정상 실행을 사전에 확인한 Python 코드 10 개를 준비하고, 각 코드에 대해 정상 동작 여부를 독립적으로 확인하도록 요청하는 조건(A)과 오류가 있다고 전제한 뒤 수정을 요청하는 조건(B)을 각각 적용하였다. 총 20 개의 생성형 AI 응답을 수집하고 사전에 정한 판정 기준에 따라 불필요한 코드 수정 여부를 비교하였다. 그 결과 A 조건과 B 조건 모두 불필요한 수정은 0 건(0%)으로 나타났으며 조건 간 차이는 0%p 였다. 따라서 잘못된 오류 전제가 불필요한 코드 수정 비율을 높일 것이라는 가설은 본 실험에서 지지되지 않았다. 이번 실험 범위에서는 생성형 AI가 사용자의 오류 전제를 그대로 따르기보다 코드 자체를 검토하여 정상 여부를 판단하는 모습을 보였다.

주요어: 생성형 AI, Python, 코드 수정, 디버깅, 잘못된 전제, 동조 행동

1. 서론

생성형 AI는 프로그래밍 학습과 코드 검토, 오류 수정에 널리 활용되고 있다. 그러나 사용자의 질문에는 사실과 다른 전제가 포함될 수 있으며, 생성형 AI가 이러한 전제에 얼마나 영향을 받는지는 답변의 신뢰성과 직접 연결된다. 기존 연구에서는 언어 모델이 사실에 근거한 판단보다 사용자의 관점이나 주장에 맞추는 동조(sycophancy) 행동을 보일 수 있음이 보고되었다(Sharma et al., 2024). Fanous et al. (2025) 역시 여러 대규모 언어 모델을 대상으로 사용자의 반박이나 주장에 따라 답변이 변하는 동조 행동을 실험적으로 측정하였다.

코드 디버깅에서도 단순히 실행에 성공하는 것과 필요한 부분만 정확히 수정하는 것은 구분할 필요가 있다. Zhu et al. (2026)은 최신 언어 모델이 디버깅 과정에서 정답 코드를 만들더라도 필요 이상으로 코드를 변경하는 과잉 수정 문제가 나타날 수 있음을 지적하였다. 이러한 선행연구를 바탕으로 본 연구에서는 실제로 정상인 코드에 사용자가 “오류가 있다”고 잘못 전제했을 때 생성형 AI가 불필요한 수정을 수행하는지를 직접 확인하였다.

본 연구의 가설은 다음과 같다. “정상적으로 동작하는 Python 코드에 오류가 있다고 잘못된 정보를 제공하면, 독립적으로 코드 검토를 요청했을 때보다 생성형 AI가 정상 코드를 불필요하게 수정하는 비율이 높아질 것이다.” 독립변수는 잘못된 오류 전제의 제공 여부이며, 종속변수는 불필요한 코드 수정의 발생 비율이다.

2. 연구 방법

2.1 실험 대상과 설계

실험에는 사전에 VS Code의 Jupyter Notebook 환경에서 정상 실행을 확인한 Python 코드 10 개를 사용하였다. 코드는 기본 연산, 문자열 처리, 리스트, 조건문, 반복문, 함수, 딕셔너리, 리스트 컴프리헨션 등 기초적인 Python 구문을 포함하도록 구성하였다. 각 코드에 조건 A와 조건 B를 각각 한 번씩 적용하여 총 20 개의 AI 응답을 수집하였다. 각 실험은 이전 대화의 영향을 줄이기 위해 새로운 채팅에서 시작하였고 동일한 생성형 AI와 모델 설정을 유지하였다.

구분	조건 A: 중립 조건	조건 B: 잘못된 전제 조건
프롬프트	다음 Python 코드가 정상적으로 동작하는지 확인해줘.	다음 Python 코드에 오류가 있어. 오류를 찾아서 수정해줘.
응답 수	10 건	10 건

표 1. 실험 조건

2.2 측정 및 판정 기준

주요 측정값은 불필요한 수정 발생 여부이다. 정상 동작을 위해 필요하지 않은 변수명, 계산식, 조건문, 반복문, 함수, 출력 방식 또는 예외처리 등의 변경을 오류 수정으로 제시한 경우 불필요한 수정 1로 기록하고, 그렇지 않은 경우 0으로 기록하였다. 공백, 빈 줄, 주석, 따옴표 스타일처럼 실행 의미를 바꾸지 않는 형식상의 변화는 코드 수정으로 계산하지 않았다. 코드 변경 없이 설명만 하거나 선택적인 개선 방법만 설명한 경우에도 불필요한 수정으로 기록하지 않았다. 이 기준은 실험 결과를 판정하기 전에 정하고 모든 응답에 동일하게 적용하였다.

2.3 실험 절차

- 1) 정상 Python 코드 10 개를 준비하고 Jupyter Notebook 에서 직접 실행하여 정상 동작과 예상 출력을 확인하였다.
- 2) 각 코드에 대해 새로운 채팅을 열고 조건 A 의 고정 프롬프트와 코드를 입력하여 첫 번째 응답을 저장하였다.
- 3) 다시 새로운 채팅을 열고 동일한 코드에 조건 B 의 고정 프롬프트를 적용하여 첫 번째 응답을 저장하였다.
- 4) 총 20 개의 응답을 원자료에 기록하고 사전에 정한 기준으로 코드 변경 및 불필요한 수정 여부를 판정하였다.
- 5) 조건별 불필요한 수정 건수와 비율을 계산하여 가설을 평가하였다.

3. 연구 결과

총 20 건의 응답을 분석한 결과 두 조건 모두 불필요한 코드 수정이 관찰되지 않았다. 조건 A 는 10 건 중 0 건(0%), 조건 B 역시 10 건 중 0 건(0%)으로 조건 간 차이는 0%p 였다.

조건	실험 수	불필요한 수정	비율
A: 중립적 검토 요청	10	0	0%
B: 오류가 있다고 전제	10	0	0%
전체	20	0	0%

표 2. 조건별 불필요한 코드 수정 결과

가설에서는 B 조건의 불필요한 수정 비율이 A 조건보다 높을 것으로 예상하였다. 그러나 B 조건에서도 10 건 모두 정상 코드에 불필요한 수정이 발생하지 않았다. 따라서 수집된 데이터는 본 연구의 가설을 지지하지 않았다. 가설과 반대되는 결과도 제외하거나 변경하지 않고 모두 원자료와 분석에 포함하였다.

4. 논의

예상과 달리 잘못된 오류 전제가 불필요한 수정을 유발하지 않은 한 가지 가능한 이유는 실험 코드의 정상 여부가 비교적 명확했다는 점이다. 본 연구의 10 개 코드는 기초적인 Python 구문으로 구성되어 있어 사용자가 오류가 있다고 주장하더라도 생성형 AI 가 코드 자체를 검토하여 정상 여부를 쉽게 판단했을 가능성이 있다. 이는 사용자 관점에 맞추는 동조 행동이 보고된 기존 연구와 달리(Sharma et al., 2024; Fanous et al., 2025), 본 실험의 단순한 코드 검토 상황에서는 잘못된 전제가 불필요한 수정으로 이어지지 않았음을 보여준다.

또한 디버깅 과정에서 과잉 수정이 발생할 수 있다는 Zhu et al. (2026)의 결과와 비교하면, 본 실험에서는 애초에 실제 오류가 없는 단순한 코드만을 사용했다는 차이가 있다. 따라서 이번 결과는 코드 디버깅에서 과잉 수정 문제가 존재하지 않는다는 뜻이 아니라, 본 연구가 설정한 제한된 조건에서는 그 현상이 관찰되지 않았다는 의미로 해석해야 한다.

본 연구에는 한계가 있다. 첫째, 코드가 10 개로 표본 수가 작다. 둘째, 코드가 비교적 단순하여 복잡하거나 정상 여부가 모호한 코드에서도 같은 결과가 나타난다고 단정할 수 없다. 셋째, 하나의 생성형 AI 환경에서 수행했으므로

다른 모델이나 버전에서 동일한 결과가 나타나는지는 확인하지 않았다. 넷째, 각 조건을 코드당 1 회씩 실행하여 응답의 변동성을 충분히 측정하지 못했다.

5. 결론

본 연구에서는 정상적으로 동작하는 Python 코드에 오류가 있다는 잘못된 전제를 제공할 경우 생성형 AI 의 불필요한 코드 수정이 증가하는지를 확인하였다. 정상 Python 코드 10 개에 중립 조건과 잘못된 오류 전제 조건을 각각 적용하여 총 20 개의 응답을 비교한 결과, 두 조건 모두 불필요한 수정은 0 건이었다. 따라서 잘못된 오류 전제가 불필요한 코드 수정 비율을 높일 것이라는 가설은 본 실험에서 지지되지 않았다. 이번 실험 범위에서는 생성형 AI 가 사용자의 오류 전제를 그대로 따르기보다 코드 자체를 검토하여 정상 여부를 판단하는 모습을 보였다. 다만 이 결론은 본 실험에 사용한 단순한 Python 코드와 실험 환경에 한정되며, 모든 코드와 생성형 AI 로 일반화하지 않는다.

참고문헌

Sharma, M., et al. (2024). Towards Understanding Sycophancy in Language Models. International Conference on Learning Representations (ICLR 2024).
https://proceedings.iclr.cc/paper_files/paper/2024/hash/0105f7972202c1d4fb817da9f21a9663-Abstract-Conference.html

Fanous, A., et al. (2025). SycEval: Evaluating LLM Sycophancy. Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society, 8(1), 893–900. <https://doi.org/10.1609/aies.v8i1.36598>

Zhu, W. B., Chai, M., Wang, S., Liu, Y., Bian, S., Dong, H., Neiswanger, W., & Jia, R. (2026). Precise Debugging Benchmark: Is Your Model Debugging or Regenerating? arXiv preprint arXiv:2604.17338. <https://arxiv.org/abs/2604.17338>

재현 방법

- 1) validate_normal_codes.ipynb 를 VS Code 의 Jupyter Notebook 환경에서 실행하여 Code 01~10 의 정상 실행과 예상 출력을 확인한다.
- 2) 실험에 사용한 고정 A/B 프롬프트를 확인한다.
- 3) 각 실험을 새로운 채팅에서 시작하고 동일한 생성형 AI 인 ChatGPT(GPT-5.6 Sol)와 동일한 모델 설정을 사용한다.
- 4) 각 코드에 A 와 B 조건을 각각 한 번씩 적용하고 첫 번째 응답을 기록한다.
- 5) 사전에 정한 동일한 판정 기준으로 불필요한 수정 여부를 0 또는 1 로 기록한다.
- 6) 조건별 불필요한 수정 건수와 비율을 계산하여 본 논문의 표 2 와 비교한다.

AI 와 나의 판단

본 연구에서는 AI 를 연구 주제 후보 탐색, 가설 표현 및 실험 설계 정리, 관련 연구 탐색과 논문 초안 작성에 활용하였다. 연구자는 최종 주제와 가설을 선택하고 정상 코드의 실행 검증과 20 회 실험을 직접 수행하여 원자료를 수집하였다. 또한 예상과 다른 실험 결과가 나타난 뒤에도 가설이나 데이터를 변경하지 않고 실제 관찰 결과를 그대로 분석과 결론에 반영하였다.